Noname manuscript No.
(will be inserted by the editor)

A Study on Analyzing Side-Channel Resistant Encoding Schemes with Respect to Fault Attacks

Jakub Breier · Dirmanto Jap · Shivam Bhasin

Received: date / Accepted: date

Abstract Recently, several software encoding countermeasures were proposed, utilizing the side-channel hiding concept for software implementations. While these schemes aim to protect the underlying code against various leakage models, they can also be utilized against fault injection attacks to some extent. This property comes from the data redundancy that is being employed in order to equalize the leakage.

In this work we analyze three different software encoding schemes with respect to fault injection attacks. We use a custom-made code analyzer to check the vulnerabilities in the assembly code and we experimentally support our results using laser fault injection technique. Our results show that implementations based on table look-up operations provide reasonable security margin and thwart fault propagation.

Keywords Fault Attacks · Software Countermeasures · Encoding Schemes · Hiding

1 Introduction

Internet-of-things and other emerging technologies are increasing our dependency on small computing devices that are used for sensing and controlling objects, remotely via network infrastructure. These devices collect and exchange data, and can often be found in critical infrastructures, controlling safety-critical processes. Therefore, bringing security into this concept is one of the major challenges and becomes a necessity in our interconnected world. Cryptographic algorithms can provide means of theoretically secure communications, preserving confidentiality and integrity of exchanged data. However, they could still be vulnerable against physical attacks, that target the implementation, which is often overlooked. Since the aim of manufacturers is to lower the price of such devices, there is often no space for comprehensive protection from physical tampering and observing implementation-specific physical information leaked from the device.

Fault injection attacks [1], considered as a popular class of physical attacks, exploit a possibility to disturb the execution of a cryptographic algorithm in order to recover information about the secret key used in the device. Evolution of fault attack methods has led to development of various countermeasures, protecting either the software implementation, hardware circuit, or the device itself from tampering.

Side-Channel attacks [7], on the other hand, utilize a different type of physical characteristic of the device. They exploit so-called *side-channel information*, which can be information about timing, power consumption, electromagnetic emanation, or other physical information that is dependent on the executed algorithm and the sensitive data.

This paper focuses on three software-based hiding countermeasures, originally proposed for side-channel protection, that can also be hardened against fault injection attacks. More specifically, we analyze a bit-sliced software countermeasure following the dual-rail precharge logic (DPL) [9], a balanced encoding scheme providing constant side-channel leakage [4], and a customized en-

J. Breier and S. Bhasin
Physical Analysis and Cryptographic Engineering
Temasek Laboratories at Nanyang Technological University
Singapore
E-mail: {jbreier,sbhasin}@ntu.edu.sg

D. Jap
School of Physical and Mathematical Sciences
Nanyang Technological University
Singapore
E-mail: djap@ntu.edu.sg

coding scheme built according to leakage model based on stochastic profiling [8]. For this purpose, we have developed a customized code analyzer that can show vulnerabilities in assembly code. Details on how such analyzers work can be further found, e.g. in [5], and implementation details of our analyzer are described in [2].

The rest of the paper is organized as follows. Section 2 provides the necessary background for our work, describing software encoding countermeasures proposed so far. Details on our custom code analyzer are stated in Section 3. Simulated and practical results of the analysis are detailed in Section 4 and 5 respectively, followed by discussion provided in Section 6. Finally, Section 7 concludes this paper and provides motivation for further work.

2 Background

The first proposal of *side-channel information hiding* in software was made by Hoogvorst et al. [6]. They suggested to adopt the dual-rail precharge logic (DPL) in the software implementation to reduce the dependence of the power consumption on the data. Their design uses a look-up table method – instead of computing the function value, the operands are concatenated and used as an address to the resulting value. The idea was explained on PRESENT implementation on AVR microcontroller.

Building on the idea of the seminal work, there were three notable works published in recent years. The rest of this section provides a short overview of each of them.

2.1 Software DPL Countermeasure

In 2013, Rauzy et al. [9] published a work that follows DPL encoding by utilizing bit-sliced technique for assembly instructions. They developed a tool that converts various instructions to a balanced DPL, according to their design. In their implementation, each byte is used to carry only one bit of information, encoded either as '01' for '1', or '10' for '0'. In the proposal, bits are chosen according to their leakage characteristics. In our work, we use the two least significant bits of the byte. This implementation uses look-up tables with balanced addressing instead of computing the operations directly. Assembly code we used in the code analysis is stated in Appendix A. For the sake of simplicity, we refer to this implementation as to the *'Static-DPL XOR'* throughout the paper.

2.2 Balanced Encoding Countermeasure

Proposed in 2014 by Chen et al. [4], this work provides assembly-level protection against side-channel attacks by balancing the number of '1's and '0's in each instruction. The code proposed by the authors is aimed for 8-bit platforms and the constant leakage is achieved by adding complementary bit to every bit of information being processed. Therefore, in each instruction, there are four effective bits of information and four balancing complementary bits. Encoding structure looks as follows: $b_3\bar{b_3}b_2\bar{b_2}b_1\bar{b_1}b_0\bar{b_0}$. Order of bits may vary depending on the leakage model. For fault injection evaluation, it does not matter which format is chosen, therefore all the data is transformed according to the structure above. Assembly code we used in the code analysis is stated in Appendix B. In [4], two basic operations are used, i.e. XOR and look-up table (LUT). For the rest of this paper, we will refer to these operations as *'Static-Encoding XOR'* and *'Static-Encoding LUT'*.

2.3 Device-Specific Encoding Countermeasure

In 2016, there was another encoding countermeasure proposal, by Maghrebi et al. [8]. The proposed encoding aims to balance the side-channel leakage by minimizing the variance of the encoded intermediate values. Previous encoding proposals were based on the assumption of Hamming Weight (HW) leakage model. However, the actual leakage model often deviates from HW, which leads to reduction in practical side-channel security of the encoding scheme. The proposal of [8] designs the encoding scheme by taking the actual leakage model into account.

The side-channel leakage is dependent on the device, and for the microcontroller case, each register leaks the information differently (though the paper argued that most of the registers have more or less similar leakage pattern). In general, the leakage normally depends on the processed intermediate value. The leakage can be formulated as follows:

$$T(x) = L(x) + \epsilon, \tag{1}$$

where L is the leakage function that maps the deterministic intermediate value (x) processed in the register to its side-channel leakage, and ϵ is the (assumed) mean-free Gaussian noise ($\epsilon \sim N(0, \sigma^2)$). The commonly used leakage function used is the n-bit representation. For example, in 8-bit microcontroller, the leakage could be represented as $L(x) = \beta_0 + \beta_1 x_1 + \ldots. \beta_8 x_8$, where x_i is the i-th bit of the intermediate value, and β_i is the i-th bit weight leakage for specific register [11]. For HW model, β_1 to β_8 are considered to be unity. In reality,

Algorithm 1: Selection of the optimal encoding function [8].

Input : m: the codeword bit-length, n: the sensitive variable bit-length, β_i: the leakage bit weights of the register, where i in $[\![1, m]\!]$
Output: 2^n codewords of m-bit length

for X *in* $[\![0, 2^m - 1]\!]$ **do**
  Compute the estimated power consumption for each codeword X and store the result in table D: $D[X] = \Sigma_{i=1}^{m} \beta_i X[i]$;
  Store the corresponding value of the codeword in the index table $I : I[X] = X$;
Sort the estimated power consumption stored in table D and the index table I accordingly
for j *in* $[\![0, 2^m - 2^n]\!]$ **do**
  Find the $argmin$ of $[\![D[j] - D[j + 2^n]]\!]$;
return 2^n codewords corresponding to $[\![I[argmin], I[argmin + 2^n]]\!]$

due to several physical device parameters, β will deviate from unity in either polarity.

The deterministic part of the leakage can then be determined as $\tilde{L} = \mathbf{A} \cdot \beta$, where $\mathbf{A} = (x_{i,j})_{1 \le i \le N; 0 \le j \le n}$, with $\mathbf{x_i}$ as a row element of $\mathbf{A}$ and N denotes the number of measurements. We can then determine $\beta = (\beta_j)_{0 \le j \le 8}$ based on the set of traces $\mathbf{T}$, as follows:

$$\beta = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{T}. \quad (2)$$

After profiling of the device to obtain the weight leakages β, the encoding function can be calculated based on the method used in Algorithm 1.

Thus, the main aim of the algorithm is to choose a set of encoding, represented as a look-up table, which minimizes the variance of the estimated power leakage. This is done by considering the leakage bit weights, which are tightly connected to the device, specifically to its registers. Hence, for different registers, different encoding setup has to be considered.

Assembly code with look-up tables is stated in Appendix C. In this paper, we will refer to this implementation as to the *'Device-Specific Encoding XOR'*, as the dependence on leakage model makes the encoding specific to a device register.

2.4 Evaluating Resistance to Fault Injection Attacks

The first two countermeasures mentioned in this section were in-depth analyzed with respect to fault injection attacks in [3]. To summarize the results, it was shown that the *Static-Encoding XOR* implementation is more vulnerable to fault attacks due to fault propagation. *Static-DPL XOR* benefits from the table look-up properties that forces the value to `0x00` every time a bad address is fetched to the loading instruction. This implementation was further analyzed and improved, making the chance of a successful attack negligible.

In this paper, we analyze the *Device-Specific Encoding XOR* implementation, compare it with previous results and we provide insights that can help designers in developing encoding schemes that are resistant against fault attacks. To make the paper self-contained, we reproduce the attacks of [3] and extend it to device-specific encoding.

3 Verification with the Code Analyzer

In order to formally verify the resistance of assembly code against fault injection attacks, we have developed a code analyzer. This section contains the design and implementation details as well as the methodology we used for analyzing the encoding implementations.

The architecture of our code analyzer is depicted in Fig. 1. Left side of the figure shows the architecture of a standard microcontroller and right side is a high-level class diagram of our analysis software written in Java language. The code analyzer is a custom instruction set simulator that is capable of simulating faults at any stage of the code execution. Time complexity of the evaluation is linear.

Program code is sent into the analyzer as a text file, following the assembly code structure. It is then broken into instructions – different subclasses of the *Instruction* class. Memory contains all the look-up tables that are statically pre-programmed instead of fetching them from a file. Registers are implemented as arrays, containing `0x00` before the program execution. *MuC* class serves as the instruction set simulator, executing instructions and performing operations on registers and memory. This class also provides the fault injection functionality – it analyzes every instruction against a chosen fault model.

The modular approach to the code analyzer improves its reusability, where it is only necessary to extend the instruction set in order to analyze a different device.

3.1 Fault Injection Analysis

The abstraction of the process of code execution and fault injection is depicted in Fig. 2. The middle part serves as a standard instruction set simulator, taking the given input, executing the code and producing the output. This process is repeated for every possible combination of inputs for every instruction and every fault model. We are analyzing resistance against four basic

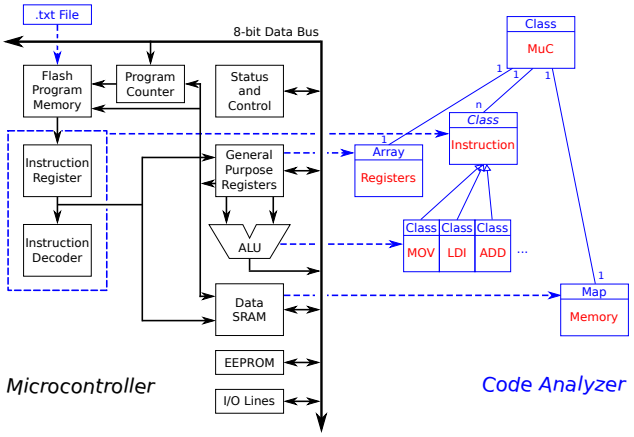


Fig. 1: Representation of microcontroller components in the Java code analyzer.

fault models: bit flip, random byte fault, instruction skip, and stuck-at fault. After the output is produced, it is analyzed by the validator which decides whether the fault changed the resulting value and if this value is useful for fault attack. We consider inputs and outputs already encoded, analyzing fault tolerance with respect to encoding/decoding is out of scope of this paper.

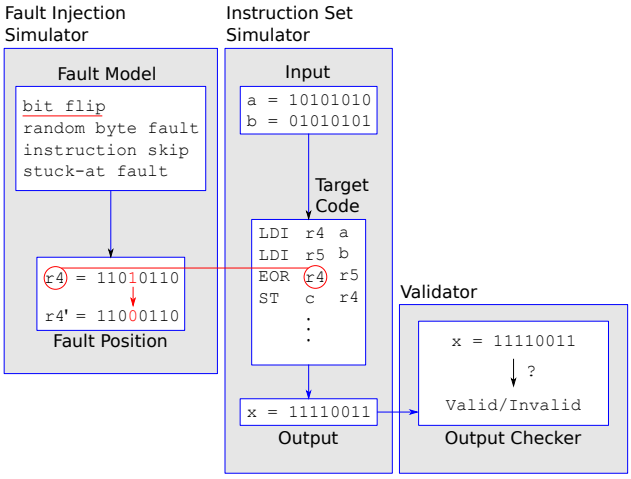


Fig. 2: Schematic of injecting a fault during the execution in the code analyzer.

In the following, we will briefly describe parts of the code analyzer:

– **Instruction Set Simulator:** As stated previously in this section, the assembly code is sent to the simulator as a text file. It accepts three different data encoding formats, according to what algorithm is currently being used. For the *Static-DPL XOR*, it accepts input in the bit-sliced complement form: $000000b_0\bar{b_0}$, therefore there are 4 possible input combinations. The *Static-Encoding XOR* accepts four bit complement format: $b_3\bar{b_3}b_2\bar{b_2}b_1\bar{b_1}b_0\bar{b_0}$, resulting to 256 input combinations. The same number of combinations is analyzed for the *Device-Specific Encoding XOR*, where the number of codewords is 16 for 8-bit code.
– **Fault Injection Simulator:** In order to get the information about algorithm resistance against fault injection, we analyze four fault models. In the case of a fault being injected into the data, we change the content of the destination register of an instruction. In bit flip fault model, we inject single and double bit flips into the *Static* countermeasures. There is no need to test other multiple bit flips, since all of them are just a subset of those two, because of DPL properties. Therefore, e.g. if an algorithm is not vulnerable against single bit flips, it will not be vulnerable against other odd-number bit flips, and vice-versa. In case of the *Device-Specific Encoding XOR*, we test all possible combinations of bit flips. Random byte fault model is a subset of bit flip fault model when it comes to code analysis, therefore this model is already included in the previous testing.
To analyze vulnerable parts against instruction skip attack, we skip either one or two instructions from the code, checking all the possible combinations. More complex instruction skip models are not considered because of the impracticability to implement them in the real environment.
Finally, to analyze the resistance against the stuck-at-fault model, we change the value of the destination register either to `0x00` or to `0xFF`.
– **Validator:** The final part of the code analyzer checks the resulting output and assigns it to one of the following pre-defined groups:
  – *VALID:* This is the most useful type of output an attacker can get. Outputs in this category follow the proper encoding of analyzed algorithm, but the value deviates from the expected value with respect to given inputs. A *VALID* fault can be directly exploited with fault injection attack.
  – *INVALID:* This type of output does not follow the encoding. Therefore, it can be easily recognized by an output checker which can then decide to discard the value instead of further propagation.
  – *NULL:* This type of fault has one of the following values: `0x00` or `0xFF`. These outputs are mostly produced by look-up table implementations and can be easily recognized as well.

4 Experimental Results for Code Analyzer

To analyze different software encoding countermeasures against fault injection attacks, we have implemented the basic operations of each previously discussed encoding scheme, i.e. *Static-DPL XOR*, *Static-Encoding XOR*, *Static-Encoding LUT* and *Device-Specific Encoding XOR* implementation. The corresponding code is provided in the appendices. As the first step, we performed a comprehensive fault analysis by putting the code under custom designed code analyzer. The main objective of this comprehensive code analysis is to uncover any native vulnerabilities in the encoding scheme under individual fault models. Such analysis cannot be done in a practical setting due to limited control over the injected fault model for a given equipement setting. Albeit it is possible to inject all the discussed fault models, it is not easy to control the fault model at will.

4.1 Code Analysis

To analyze vulnerabilities in different schemes, the basic operations were fed to the code analyzer. The analyzer considers 3 different fault models, i.e. stuck-at, bit flips and instruction skip. Both single and multiple bit-flips are possible. The first three analysed operations i.e. *Static-DPL XOR*, *Static-Encoding XOR*, *Static-Encoding LUT* are a special case, where more than 2-bit flips are equivalent to 1-bit or 2-bit flips eventually. The analyzer reports the impact on the final output in presence of discussed fault models. This is represented as a normalized distribution of faulty output for each considered fault models. Three outputs are expected: `VALID`, `INVALID` and `NULL`. `VALID` implies that final faulty output stays within the encoding. Similarly, `INVALID` refers to the faulty output which is no longer in the applied encoding. `NULL` faults are `0x00` or `0xFF` values at the output. While `VALID` faults stay within the encoding and can lead to differential fault analysis (DFA), it is rather less likely with `INVALID` faults. On the other hand, `NULL` deletes any data dependent information, disabling any further exploitation by DFA. *Therefore, `VALID` faults must be prevented at all costs, while keeping `INVALID` in check and maximizing `NULL` faults.* The analysis results for Static-DPL XOR, Static-Encoding XOR, Static-Encoding LUT are shown in Fig. 3. We discuss each of the results in the following.

The fault distribution of *Static-Encoding XOR* is shown in Fig. 3 (a). This encoded operation does not produce any `VALID` faults for 1-instruction skip and 1-bit flip. The percentages of `VALID` faults for other fault models stay between 4-6%. Majority of the faults (92-100%) result in `INVALID` faults while only double

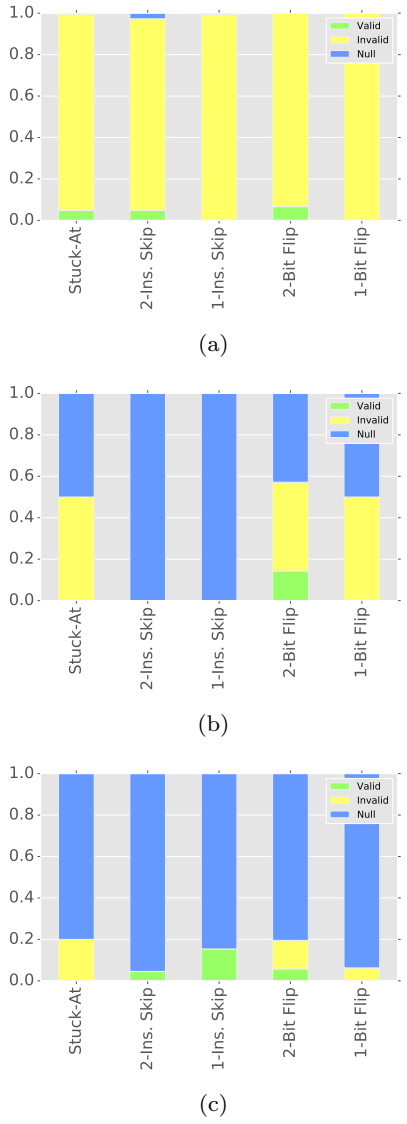


Fig. 3: Fault distributions of (a) *Static-Encoding XOR*,(b) *Static-Encoding LUT*, (c) *Static-DPL XOR* code analysis.

instruction skip result in a non-negligible `NULL` faults (2.7%). Although `INVALID` faults are more desirable than `VALID` faults, later we will show that some `INVALID` can be exploitable in particular for *Static-Encoding XOR*.

The *Static-Encoding LUT* shows an altogether different fault resistance (Fig. 3 (b)). This encoded operation produces much more `NULL` faults than the previous case, which is a desirable property. Instruction skips result in 100% `NULL` faults, while stuck-at and 1-bit flips produce 50% `INVALID` and 50% `NULL` faults. The only way to produce `VALID` faults in this operation is to inject 2-bit flips which result in 14.2% `VALID` faults. Rest

of the faults would result in `INVALID` or `NULL` faults with equal probability.

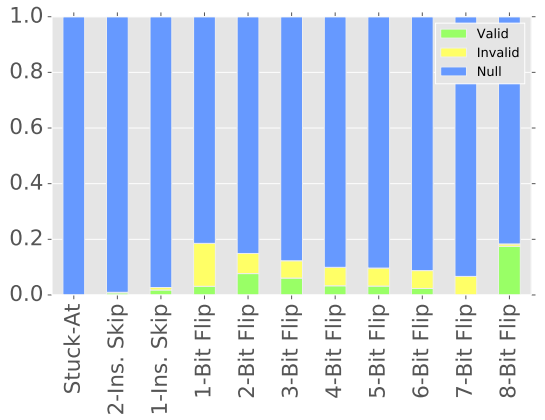


Fig. 4: Fault distributions of *Device-Specific Encoding XOR* code analysis.

The analysis results of *Static-DPL XOR* are shown in Fig. 3 (c). While no `VALID` faults are possible for stuck-at and 1-bit flip, it stays below 6% for 1-instruction skips and 2-bit flips. The worst performance is under 2-instruction skip model, where the percentage of `VALID` faults is as high as 15.3%. The high vulnerability against 2-bit flips can be explained by the fact, that 2-bit flips are the limit of the dual-rail encoding scheme. Apart from these, the other faults are more likely to be `NULL` rather than `INVALID`, which is desirable.

Finally we applied the code analysis on *Device - Specific Encoding XOR* operation as shown in Fig. 4. The difference from previous cases is that, here multi-bit flips cannot be dealt as a subset of 1-bit and 2-bit flips. Therefore the analysis covers bit flips from 1-bit to 8-bits, i.e. the data width of the target processor. It can be easily observed that this encoding scheme is more likely to produce `NULL` faults which is highly desirable. For the `VALID` faults, stuck-at model produce none, while only $< 2\%$ can be achieved by instruction skips. In case of bit flips, the percentage of `VALID` faults stays between 2-7% with the exception of 7 and 8 bit flips. However, for different β coefficients used in the leakage function, results on bit flips should be slightly different, but consistent with the expectations. The total value of *VALID* bit flips for all the possibilities range within 4.2-4.7%, but their distribution is different, depending on used coefficient.

5 Experimental Evaluations on Atmel Platform

Following the code analysis, the fault resistance was experimentally verified. In this step, the corresponding code was implemented on a real AVR microcontroller and tested under laser fault injection. The objective of practical validation was to find which of the known vulnerabilities are producible with equipement at hand.

5.1 Chip Decapsulation

In order to conduct the experiments, the microcontroller had to be decapsulated. The microcontroller was de-packaged from the backside, using a precise milling equipment. While performing this process, there had to be several pauses in between the grinding sessions, to let the chip cool down. First, the epoxy layer had to be milled down. Then, the copper substrate was thinned, so that it could be peeled off. As the thinning process came to an end, there was still glue which is supposed to hold the chip on the copper substrate. This was removed by hard plastic tools. Finally, we obtained a sample ready for the experiments. One can still use more advanced milling/polishing tools in order to thinner the silicon substrate and make the surface even.

5.2 Laser Setup

The fault injection was done with a near-infrared diode pulse laser with a pulse power of 20 W (reduced to 8 W with 20× objective). The pulse repetition rate of the laser is 10MHz and spot size is 30×12 μm (15×3.5 μm with 20× objective). Intentional `nop` were inserted at beginning of each node to overcome the 100 ns delay between trigger and laser injection. The target platform was Atmel ATmega328P microcontroller, de-packaged and mounted on Arduino UNO development board. The surface area of the chip is 3x3 mm^2, manufactured in 350 nm CMOS technology. An X-Y positioning table with a step precision 0.05 μm was used to scan the chip surface and perform laser injection. The timing of injection was synchronised with executed code using a code-generated trigger. The injection platform along with the target is shown in Fig. 5.

5.3 Laser Experiments Results

The prime difference from the previous analysis is that in the experimental validation, we cannot precisely control the fault model. Therefore, the fault models are not uniformly distributed. Before starting the real experiment, we performed some profiling on the target with

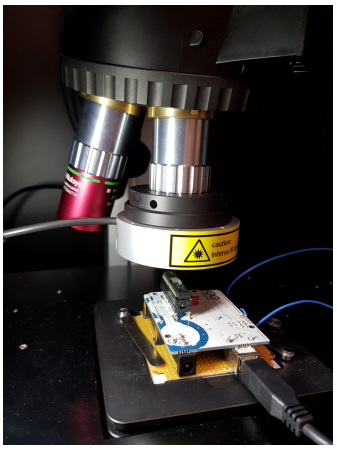


Fig. 5: ATmega328P device under a near-infrared diode laser injection setup.

basic assembly code and verified that all the fault models are possible to produce experimentally. Next, we flash the assembly code of the four previously discussed software encoding operations. We essentially note the percentage of `VALID`, `INVALID` and `NULL` faults produced for each tested operation. The results are summarised in Fig. 6.

Static-Encoding XOR shows the best consistency with the simulated analysis previously (see Fig. 6 (a)). While 93.56% of the faults are `INVALID`, only 5.88% `VALID` were produced. Moving towards *Static-Encoding LUT*, we observe a 32.42% `VALID` faults in Fig. 6 (b). Since a `VALID` fault in this implementation can only result from even bit flips, this infers that the fault model distribution is biased towards multiple bit flips in our experiments. Similarly, we also observe a 22.2% `VALID` faults in *Static-DPL XOR* (Fig. 6 (c)) owing to the prevalent multiple bit flip model.

When it comes to *Device-Specific Encoding XOR* (Fig. 6 (d)), results show distribution very similar to the one obtained by the code analysis. Because it is more likely to produce bit flips when injecting faults in the microcontroller, at 13.5% an inflated number of *VALID* faults can be observed in this case, with a relatively small number of *INVALID* faults. As expected, *NULL* outputs are dominant i.e. 82.5% , because of the look-up table properties.

6 Discussion

In this section, we will discuss some important parameters of particular encoding implementation with respect to fault injection attacks.

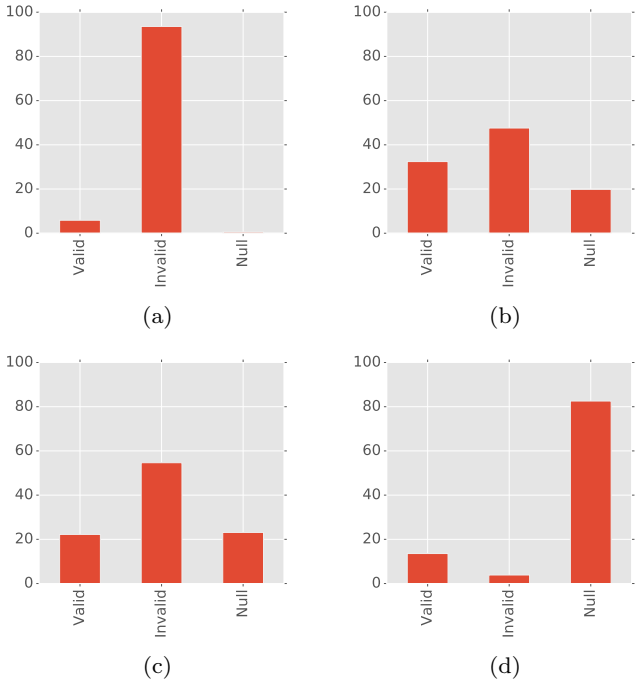


Fig. 6: Fault distributions of (a) *Static-Encoding XOR*,(b) *Static-Encoding LUT*, (c) *Static-DPL XOR*, and (d) *Device-Specific Encoding XOR* experiments.

6.1 Selection of β Coefficients

We considered several parameters for the code analysis of *Device-Specific Encoding XOR*. We analyzed different β values scenarios. We considered the case where the variance of the β is relatively high (the βs might be cancelling each other), and the case where the variance of the β is low (almost Hamming weight).

The most significant difference can be observed in the result for implementation with β coefficients that do not follow Hamming weight leakage model (stated in Fig. 7 (a)). From the figure, it can be observed that the number of 1-bit flips is inflated, compared to the almost Hamming weight case (stated in Fig. 7 (b)). The behavior of the faults shows contrast between different beta values, which is not the case for other encoding schemes, and hence could be further investigated.

6.2 Fault Propagation

When considering security of different implementations, fault propagation is an important factor that can significantly affect the possibility to mount an attack. In case we want to prevent a successful fault attack, it is necessary to avoid the propagation of an *INVALID* output when it is fed as an input to a next iteration of

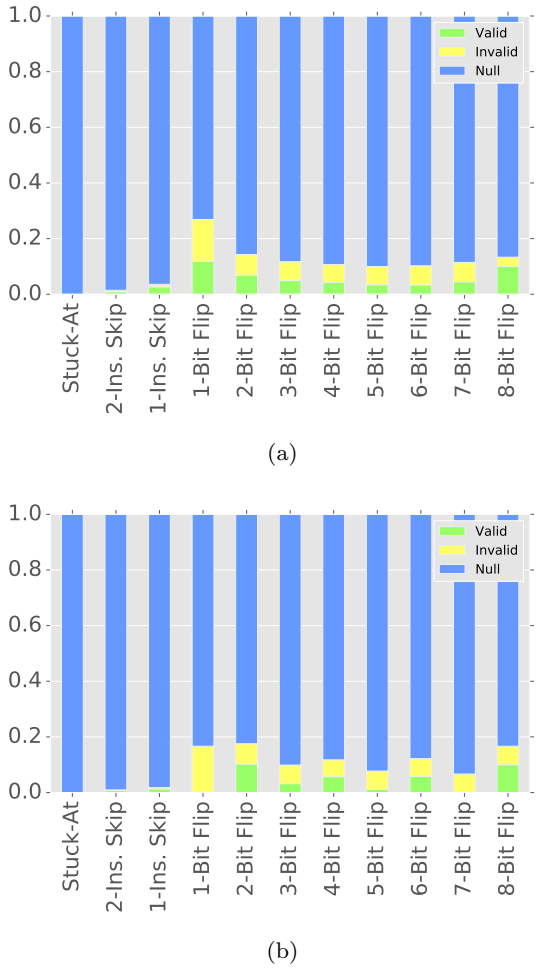


Fig. 7: Fault distributions of *Device-Specific Encoding XOR* code analysis with (a) high variance and (b) almost Hamming weight

the algorithm. Otherwise, this output could leak some information about the processed data and therefore, allow an attacker to use the differential fault analysis.

From this point of view, look-up table implementations have an advantage, since every input that does not follow the encoding will be automatically converted to *NULL*. Analysis results of *Static-DPL XOR*, *Static-Encoding LUT*, and *Device-Specific Encoding XOR* show that if any of the inputs is either *INVALID* or *NULL*, it will always output *NULL*. Situation with the *Static-Encoding XOR* is different because of the algorithm design. There are several combinations of inputs that lead to *VALID* faults – more specifically, any combination of:

– Two *INVALID* inputs,
– Two *NULL* inputs,
– *INVALID* and *NULL* inputs.

Moreover, for a combination of *VALID* and *NULL* inputs leaks a complete information about the *VALID* input in the form $\bar{v_3}\bar{v_3}\bar{v_2}\bar{v_2}\bar{v_1}\bar{v_1}\bar{v_0}\bar{v_0}$, where $v_3v_2v_1v_0$ is the original input.

To summarize, table look-up implementations provide solid protection against fault attacks when it comes to fault propagation. Any other implementation that uses standard operations performed by using ALU, can be vulnerable if it is not directly designed with such goal in mind. Therefore, when designing a fault resistant algorithms along with the side-channel resistance, look-up tables can offer fault propagation cancellation by default.

6.3 Accuracy of the Code Analyzer

After observing the experimental results, one can notice the (sometimes significant) difference, compared to simulated results. As we already mentioned in Section 5, when performing the expriment, the attacker often cannot precisely control the fault model. Moreover, different fault injection techniques lead to various outcome on different devices under test.

The purpose of the code analyzer is to show possible vulnerabilities with respect to provided code. Once the probability of inducing a *VALID* fault into the code is greater than zero, it means the attacker will always be able to inject a fault if he chooses a proper fault injection equipment and uses correct settings, unless there is some other tamper protection implemented in the device.

On the other hand, if the probability of a *VALID* fault is zero in the simulated environment, it stays the same in the experimental settings, no matter what technique the attacker uses.

6.4 Simulating a Pipelined Architecture

When targeting a pipelined architecture, the fault can be injected in all the instructions in the pipeline at a current time. For example, Fig. 8 depicts a 4-stage pipeline, where a fault was injected at cycle 4. The fault can disturb up to four instructions, each at a different stage, however, in this case, only the instruction in the *Execute* stage was disturbed. As observed in [12], different pipeline stages can have a different fault sensitivity. An advanced attacker may be able to determine the sensitivity threshold by carefully profiling the device, making it possible to precisely target each stage. Another observation in [12] is that the nature of the fault is dependent on the stage. In the 7-stage RISC processor they used, disturbing the first two stages can cause

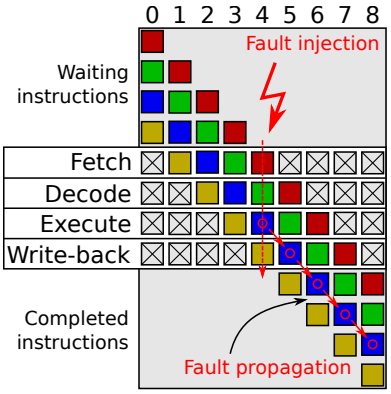


Fig. 8: Fault injection on a 4-stage pipelined architecture.

instruction skips, while the later stages can cause data faults.

Crafting an analyzer with a pipeline fault simulation is possible, but it would require a deep analysis of the target platform and the code analysis result would be dependent on this platform. Therefore, it would not be possible to get universal result with respect to given fault models.

7 Conclusion

This paper summarizes fault attack resistance of three software-based encoding schemes that were introduced to prevent side-channel attacks. We mainly aim at analyzing the *Device-Specific Encoding XOR* implementation that was proposed recently. Our results provide insights and comparison against the other schemes (*'Static'* encoding schemes) that have been analyzed in previous work [3].

In general, table look-up schemes offer higher level of security by thwarting the fault propagation in case of several algorithm iterations. *Static-Encoding XOR* might look the best from the experimental results, however, fault propagation properties of this design allow attacker to easily mount a DFA attack or to directly observe inputs passed to the algorithm in case of other input being of a *NULL* type. After considering this phenomenon together with code analysis and experimental results, we conclude that the *Device-Specific Encoding XOR* is currently the most secure scheme with respect to fault attacks and provides a decent level of security.

For the future work, we would like to extend the code analyzer to support pipelined architectures, being able to discover vulnerabilities w.r.t. more comprehensive fault models, like cache attacks (e.g. as described in [10]).

References

1. Biham, E., Shamir, A.: Differential Fault Analysis of Secret Key Cryptosystems. In: CRYPTO '97, *LNCS*, vol. 1294, pp. 513–525 (1997)
2. Breier, J.: On analyzing program behavior under fault injection attacks. In: 2016 11th International Conference on Availability, Reliability and Security (ARES), pp. 474–479 (2016). DOI 10.1109/ARES.2016.4
3. Breier, J., Jap, D., Bhasin, S.: The other side of the coin: Analyzing software encoding schemes against fault injection attacks. In: 2016 IEEE International Symposium on Hardware Oriented Security and Trust (HOST), pp. 209–216. IEEE (2016)
4. Chen, C., Eisenbarth, T., Shahverdi, A., Ye, X.: Balanced Encoding to Mitigate Power Analysis: A Case Study. In: CARDIS, Lecture Notes in Computer Science. Springer (2014). Paris, France
5. Dureuil, L., Potet, M.L., de Choudens, P., Dumas, C., Clédière, J.: From Code Review to Fault Injection Attacks: Filling the Gap Using Fault Model Inference, pp. 107–124. Springer International Publishing, Cham (2016)
6. Hoogvorst, P., Danger, J.L., Duc, G.: Software Implementation of Dual-Rail Representation. In: COSADE (2011). Darmstadt, Germany
7. Kocher, P.C., Jaffe, J., Jun, B.: Differential power analysis. In: Proceedings of the 19th Annual International Cryptology Conference on Advances in Cryptology, CRYPTO '99, pp. 388–397. Springer-Verlag, London, UK, UK (1999)
8. Maghrebi, H., Servant, V., Bringer, J.: There is wisdom in harnessing the strengths of your enemy: Customized encoding to thwart side-channel attacks – extended version. Cryptology ePrint Archive, Report 2016/183 (2016)
9. Rauzy, P., Guilley, S., Najm, Z.: Formally Proved Security of Assembly Code Against Leakage. IACR Cryptology ePrint Archive **2013**, 554 (2013)
10. Rivière, L., Najm, Z., Rauzy, P., Danger, J.L., Bringer, J., Sauvage, L.: High precision fault injections on the instruction cache of ARMv7-M architectures. In: Hardware Oriented Security and Trust (HOST), 2015 IEEE International Symposium on, pp. 62–67 (2015). DOI 10.1109/HST.2015.7140238
11. Schindler, W., Lemke, K., Paar, C.: A stochastic model for differential side channel cryptanalysis. In: International Workshop on Cryptographic Hardware and Embedded Systems, pp. 30–46. Springer Berlin Heidelberg (2005)
12. Yuce, B., Ghalaty, N.F., Schaumont, P.: Improving fault attacks on embedded software using risc pipeline characterization. In: 2015 Workshop on Fault Diagnosis and Tolerance in Cryptography (FDTC), pp. 97–108 (2015). DOI 10.1109/FDTC.2015.16

A Assembly Code For *Static-DPL XOR* Implementation

Table in this section contains assembly code used for the code analysis. Note that there are several differences in comparison to the original paper. We precharge all the registers before the code execution, therefore there is no need to use precharge instructions. The other change is in instructions 7 and 8, where we first load the operation code (can take values `01010101` for `and`, `10101010` for `or`, and 01100110 for `xor`) and then we execute `ldd` instruction using the destination register, operation code and value. Look-up tables are stated in Table 2.

Table 1: Assembly code for *DPL XOR* in AVR

#	Instruction	#	Instruction
0	ldi r1 *a*	5	andi r2 00000011
1	ldi r2 *b*	6	or r1 r2
2	andi r1 00000011	7	ldi r4 *operation*
3	lsl r1 1	8	ldd r3 r4 r1
4	lsl r1 1	9	mov d r3

Table 2: Look-up tables for `and`, `or`, and `xor`

index	and	or	xor
0000 - 0100	00	00	00
0101	01	01	10
0110	10	01	01
0111 - 1000	00	00	00
1001	10	01	01
1010	01	10	10
1011 - 1111	00	00	00

B Assembly Code for *Static-Encoding XOR* Implementation

The code stated in Tab. 3 follows the originally proposed algorithm for *Static-Encoding XOR*. This implementation uses several constants, either for clearing and precharging the registers before loading the data (e.g. `ldi r16 11110000`), or for changing the data to proper encoding format (e.g. `ldi r17 01011010`).

Table 3: Assembly code for *Encoding XOR* in AVR

#	Instruction	#	Instruction
0	ldi r1 *a*	19	and r20 r1
1	ldi r2 *b*	20	and r21 r1
2	ldi r16 11110000	21	swap r21
3	ldi r17 11110000	22	or r20 r21
4	and r16 r1	23	ldi r22 00001111
5	and r17 r1	24	ldi r23 00001111
6	swap r17	25	and r22 r2
7	or r16 r17	26	and r23 r2
8	ldi r18 11110000	27	swap r23
9	ldi r19 11110000	28	or r22 r23
10	and r18 r2	29	ldi r21 10100101
11	and r19 r2	30	eor r20 r21
12	swap r19	31	eor r20 r22
13	or r18 r19	32	ldi r24 11110000
14	ldi r17 01011010	33	ldi r25 11110000
15	eor r16 r17	34	and r24 r16
16	eor r16 r18	35	and r25 r20
17	ldi r20 00001111	36	or r24 r25
18	ldi r21 00001111		

C Assembly Code for *Device-Specific Encoding XOR* Implementation

In this section, we describe the code used for *Device-Specific Encoding XOR*. After determining the bit leakage weights, and computing the encoding based on Algorithm 1, several look up tables are constructed.

According to the original paper [8], it is reasonable to split an n-bit variable into two different halves in order to avoid holding large look-up tables in memory. Therefore, we use two registers for processing each value.

In Table 4, the pseudocode for the encoding is presented. First, the upper nibble is retrieved for input a and b (a_h and b_h) under the encoding format ($f(a_h)$ and $f(b_h)$), using the *luthb* table, followed by the lookup table *lutop* used to perform xor operation ($LUT(f(a_h) << 4||f(b_h)) = f(a_h \oplus b_h)$). Similar procedure is done for the lower nibble, using the *lutlb*.

Table 4: Assembly pseudocode for *Device-Specific Encoding XOR* in 8-bit AVR.

#	Instruction	#	Instruction
1	ldi r1 *a*	12	eor r4 r4
2	ldi r2 *b*	13	ldd r4 *lutlb* r1
3	eor r3 r3	14	eor r5 r5
4	ldd r3 *luthb* r1	15	ldd r5 *lutshift* r4
5	eor r4 r4	16	eor r6 r6
6	ldd r4 *lutshift* r3	17	ldd r6 *lutlb* r2
7	eor r5 r5	18	or r5 r6
8	ldd r5 *luthb* r2	19	eor r4 r4
9	or r5 r4	20	ldd r4 *lutop* r5
10	eor r3 r3		
11	ldd r3 *lutop* r5		